TECHNICAL DOCUMENTATION

NL → SQL Multi-Agent Analytics Assistant (API + Streamlit + Local LLM Pipeline)

1. Introduction

This document provides a complete technical overview of the NL→SQL Analytics Assistant that converts natural language questions into validated Oracle SQL queries, executes them (optional), and produces downloadable analytic reports. The system is designed with two operational modes:

1. **API-driven architecture**:
   A FastAPI backend orchestrating LLM agents, RAG, SQL execution, and reporting; combined with a Streamlit frontend that communicates with the backend via REST endpoints.
2. **Standalone Streamlit architecture (app.py)**:
   A compact, single-file flow combining the entire pipeline—agents, RAG, SQL execution, and UI—within one process.
   This mode is primarily useful for debugging, local testing, and demos without running a backend server.

The system supports both cloud LLMs and **local LLM inference** (LLAMA, Mixtral, Qwen) and integrates retrieval-augmented generation (RAG) via FAISS indexing. Outputs include validated SQL, query results, and multi-format reports (HTML, PDF, CSV, DOCX).

---

2. System Overview

At its core, the system implements a **multi-agent orchestration pipeline** that transforms a natural-language business question into an executable Oracle SQL query. The steps include:

1. **Intent Classification (Agent-0)**
2. **Context Building via RAG (FAISS Retrieval)**
3. **Query Planning (Agent-1)**
4. **SQL Composition (Agent-2)**
5. **SQL Verification and Refinement (Agent-3)**
6. **Oracle Execution (optional)**
7. **Report Generation (optional)**

These components are wired together differently depending on whether the system runs:

- Via **API + Streamlit frontend**, or
- Directly via **standalone app.py**.

The following sections explain each piece in detail.

---

3. Architecture

3.1 API-Driven Architecture (api.py + frontend.py)

Backend (FastAPI)

The backend acts as the central orchestrator. It exposes well-structured REST endpoints:

- /health — system heartbeat
- /v1/intent — runs Agent-0
- /v1/pipeline/run — executes the full multi-agent pipeline
- /v1/sql/execute — optionally runs validated SQL against Oracle
- /v1/report — renders and exports report formats

Each endpoint delegates tasks to modular service layers:

- agents/ for LLM execution
- retrieve_context.py for RAG
- oracle_exec.py for database queries
- report_utils.py and templates for rendering

The backend supports **local model inference** through the embeddings and local LLM loader modules.

Frontend (Streamlit)

frontend.py operates purely as a UI layer. It does not contain business logic; instead, it:

1. Accepts user natural-language questions.
2. Sends them to the backend via REST.
3. Receives SQL + results.
4. Displays JSON outputs, SQL, tables.
5. Provides download buttons for generated reports.

This decoupling ensures scalability, easy deployment, and independent development of UI.

3.2 Standalone Architecture (app.py)

In this mode:

- The Streamlit UI,
- Multi-agent logic,
- RAG,
- SQL execution, and
- Reporting

all run inside a single file, without a backend.

This mode directly imports:

- agent0_intent_llm
- agent1_planner_llm
- agent2_composer_llm
- agent3_verifier_refiner_llm
- retrieve_context
- oracle_exec
- local LLM loader

The entire pipeline is executed in-memory and sequentially.

---

4. Multi-Agent Pipeline

4.1 Agent-0: Intent Classifier

Agent-0 determines whether the user's input is:

- A valid analytic question
- A conversational or irrelevant message
- A harmful/unsafe query
- Something requiring clarification.

It outputs:

{
"intent": "sql_query",
"should_run_pipeline": true,
"assistant_reply": null,
"reason": "User asked for metrics over trade data"
}

This step prevents invalid or risky prompts from progressing through the SQL pipeline.

4.2 Retrieval-Augmented Generation (RAG)

The RAG layer supplies schema-relevant context to Agents 1–3. It uses:

- schema_card.md
- fewshots.docs.jsonl
- glossary.docs.jsonl
- domain documents (if provided)

RAG Process:

1. Embed the user query using local embedding models.
2. Search FAISS index
3. Return relevant glossary, few-shots, schema segments.
4. Produce a context bundle fed to Agent-1

The design ensures SQL generation stays grounded in real schema definitions.

4.3 Agent-1: Query Planner

Using the RAG context + user question, Agent-1 constructs a **JSON "planning envelope"**, which defines the structure of the SQL query:

- Required tables.
- Required joins.

- Filters + metrics
- Date logic
- Grouping/aggregation
- Sorting
- Row limiting
- Special business rules

This envelope serves as a machine-readable blueprint for SQL composition.

4.4 Agent-2: SQL Composer

Agent-2 transforms the planning envelope into actual **Oracle SQL text**. Responsibilities:

- Join construction.
- Column resolution
- Oracle-specific functions
- Aggregation
- WHERE logic
- Time window conversions
- Ordering and row limiting
- Fully qualified table names

It returns to the SQL string that is ready for validation.

4.5 Agent-3: SQL Verifier & Refiner

The final agent validates SQL generated by Agent-2. It performs:

1. Syntax review
2. Semantic cross-checks
3. Reference validation (columns, tables)
4. Logical sanity checks
5. Oracle compatibility adjustments

If necessary, it rewrites or patches the SQL to reach a clean, reliable final version.

Output example:

```
{
  "SQL": "SELECT ...",
  "rationale": "SQL validated; owner-qualified tables added."
}
```

---

5. Retrieval Layer (FAISS + Embeddings)

The retrieval subsystem uses:

- Local embedding models (in embeddings/ and model directories)
- FAISS index files stored under the data directory.
- A registry file mapping documents to their embeddings

Key technical points:

- Supports incremental FAISS updates.
- Uses sentence embeddings optimized for semantic similarity.
- Retrieval adjustable by top-k and threshold parameters
- Integrates cleanly with all agents.

This layer enables schema grounding, pattern-based generation, and contextual accuracy.

---

6. Oracle SQL Execution Layer

The DB engine is located inside db./oracle_exec.py.

Capabilities:

- Secure connection creation using .env variables.
- Exception-safe execution wrappers
- Automatic row limiting
- Result shaping into JSON and Pandas Data Frames
- Connection pooling compatibility

- Read-only execution recommended

The SQL executor is used only when:

run_sql: true in /v1/pipeline/run
or
the user triggers execution in the Streamlit UI.

---

7. Reporting Engine

The reporting subsystem includes:

- Jinja2 HTML template (report.html)
- Conversion into:
    - HTML
    - PDF (wkhtmltopdf)
    - CSV
    - Excel
    - DOCX

Workflow:

1. SQL results are formatted into tables
2. Template is populated with:
    - Summary section
    - Query metadata
    - Result tables
3. Optional charts/visuals included
4. Converted to the requested output format

The /v1/report API endpoint bundles these into downloadable files.

---

8. Local LLM Models (LLAMA, Mixtral, Qwen, Others)

Your code dump includes a **complete local model integration layer**, capable of running:

- GGUF quantized models

- Mixtral variants
- Llama 3 / 2 derivatives
- Qwen
- Custom local models

Loader Capabilities:

- Automatic GPU/CPU selection
- Model caching
- Thread-level configuration
- Token streaming
- Embeddings and inference separation

The system can switch between local and remote models depending on configuration.

---

9. API Endpoints

9.1 /v1/intent

Input:

{ "query": "..." }

Output:
Intent classification JSON.
Used by frontend before deciding next steps.

9.2 /v1/pipeline/run

Primary pipeline endpoint.

Input example:

```
{
  "query": "Show total trade notional by broker for last month",
  "options": {
    "execute_sql": false,
    "generate_report": false
```

}

}

Output contains:

- Agent-0 → Intent
- RAG → Context bundle
- Agent-1 → Planning envelope
- Agent-2 → Draft SQL
- Agent-3 → Final SQL
- (Optional) DB results
- (Optional) Report file paths

9.3 /v1/sql/execute

Executes arbitrary validated SQL.

Input:
SQL string + execution options.

Output:
Result rows + metadata.

9.4 /v1/report

Builds report files from:

- SQL
- DataFrame
- Rendering options

Outputs binary files for frontend download.

---

10. Frontend (Streamlit UI)

The Streamlit frontend communicates purely via the API.

Primary components:

- Query input box
- SQL preview
- Agent output sections
- Result table viewer
- Report download buttons
- System logs (optional)

The frontend does not run any AI logic; it simply orchestrates user interaction.

---

11. Logging, Error Handling & Utilities

Your codebase includes utility modules for:

- JSON normalization
- Prompt construction
- Logging standardization
- Consistent error responses from all pipeline stages
- Safe execution wrappers
- Request/response validation

These ensure predictable behavior across both operational modes.

---

12. Differences Between the Two Modes of Operation

Feature	API + Frontend	Standalone app.py
LLM Pipeline	Runs in backend	Runs in same file
Streamlit UI	Uses REST calls	Direct call to pipeline
Deployment	Multi-service	Single process
Scalability	High	Limited
Debugging	Clean per-agent logs	Easier to inspect inline

---

13. Environment & Configuration

Key .env variables include:

- Oracle DSN / User / Password
- Local LLM model paths
- Embedding model selection
- FAISS index paths
- wkhtmltopdf path
- Logging level

The system loads these at startup for both architectures.

14. Extensibility

The architecture supports:

- Adding more agents (e.g., post-analysis agent)
- Supporting multiple database types
- Adding new report templates
- Expanding RAG documents
- Swapping local models
- Adding monitoring hooks

The modular file structure makes extension predictable and maintainable.

15. Conclusion

The NL→SQL Analytics Assistant is a fully modular, multi-agent, RAG-enhanced system capable of translating natural language into validated Oracle SQL, executing queries, and generating professional-grade analytics reports. The dual-architecture design provides flexibility for production deployment (API + Frontend) and easy local experimentation (app.py).

This document covers the entire technical landscape of the system without diving into unnecessary per-file detail, while ensuring the full pipeline and all major components are deeply and accurately represented